

Smart City Traffic Intelligence

Part 3, Task 7: Responsible and Sustainable AI

Bias and Fairness Report, with a Governance and Sustainability Discussion

Gautam K Das

This document fulfils Task 7 of the capstone. It covers two things:

- a bias and fairness report for the models and systems built across Part 3, and
- a short discussion of governance and sustainability in mobility AI.

Rather than treating these as abstract or generic risks, this report calls out specific, concrete points that surfaced during the actual development of this project, referencing the exact figures, scripts and decisions where they occurred.

1. Bias and Fairness Report

1.1 Sampling and coverage limitations

- Single road, single direction, single city: the dataset covers only westbound traffic on I-94 near Minneapolis and Saint Paul. Nothing in this project has been validated against any other road, the opposite direction of the same road, or any other city, and **none of the models or recommendations should be assumed to extend or generalise beyond this one corridor.**
- A six-year window that is now several years old: the data runs from October 2012 to September 2018. Traffic patterns can shift for reasons the dataset cannot capture, changes to road capacity, population growth, or broader shifts in commuting behaviour, **so any use of these models against present day conditions should be treated as provisional until thoroughly checked and stress tested against more recent data.**
- Missing hours are substantial and their cause is unknown: **the gap analysis carried out for the LSTM demand model (Task 3) found that roughly 23% of all hours in the six-year window have no record at all.** It is not known whether these gaps happen at random or are concentrated in particular conditions. If, for example, sensor outages are more likely during severe storms, then the severe weather hours that do survive in this dataset would already be a skewed sample of severe weather rather than a representative one, and the data alone cannot rule this out.
- Cleaning corrected known errors rather than filled genuine gaps: pipeline.py removed 17 exact duplicate rows, collapsed 5,430 duplicate timestamp hours down to one row per hour, **found 10 rows with a physically impossible temperature at or below zero Kelvin, and found one hour with an implausible rainfall reading of 9,831.3 millimetres.** These were corrected or imputed from a monthly median rather than dropped, which keeps the dataset's overall shape but means a small number of hours carry an imputed rather than an observed value.
- Rare weather categories are thin on evidence: **two of the eleven weather_main categories, Smoke and Squall, occur too rarely in the data** for the Task 5 recommendation system to produce a reliable time of day recommendation, a limitation documented directly in traffic_recommendation.py rather than smoothed over. The same imbalance runs through every model that uses weather_main: **results for common conditions such as Clear or Clouds rest on far more evidence than results for rare conditions such as Thunderstorm or Squall.**
- Holidays are especially sparse: **only 53 of 40,575 rows, about 0.13%, are flagged as a US federal holiday.** Any model behaviour that is specific to holidays is being learned from a very small number of examples, a point the deployment testing in Task 6.3 ran into directly when a single holiday request produced a noticeably different prediction from otherwise similar non holiday requests.

1.2 Proxy labels and their risks

No real accident dataset was available for this project. The `high_risk` classification target used throughout Part 3 is a proxy, defined in `data_prep.py` as high or severe traffic congestion, the top half of the traffic volume distribution by quartile, occurring at the same time as severe weather (Thunderstorm or Squall) or low visibility conditions (Fog, Mist, Haze or Smoke). Under this definition, 1,926 of 40,575 rows, 4.75%, are flagged `high_risk`.

This proxy encodes an assumption rather than a measurement: that congestion combined with bad weather is a reasonable stand in for elevated accident risk. That assumption has not been tested against real collision records, and there are plausible reasons it could be wrong in either direction. Congestion often comes with lower speeds, which is sometimes associated with fewer severe injuries even if minor collisions become more frequent, while some of the more serious accidents happen in clear conditions at higher speed. **A model trained on this proxy is really learning when congestion coincides with bad weather, not when an accident is likely, and the two are not guaranteed to move together.**

A more specific, structural risk is worth flagging plainly. The classifier's own input features, `FEATURE_COLS` in `supervised_models.py`, include `is_severe_weather`, `is_low_visibility`, and one hot encodings of `weather_main`, the same weather categories used to define the `high_risk` label in the first place. **This is not a hypothetical risk: when this classifier was actually trained, `is_low_visibility` came out as its single largest feature importance at 0.372, over a third of the total, with `wx_Mist`, `is_severe_weather` and `wx_Thunderstorm` also ranking in its top eight.** Part of what looks like the model learning to predict risk is therefore, unavoidably, the model learning to reconstruct its own label definition from features that closely restate that definition. The classifier's reported accuracy and feature importances should be read with that circularity in mind, not treated as independent evidence that these weather features are genuinely predictive of accident risk.

A related caution came up directly during the recommendation system's development. **The hourly traffic profile used to cross check the recommended time windows in Task 5 was itself built from the same random forest regressor the recommendations are based on, which is why the correlation between the two came back close to perfect the first time it was checked and prompted a direct question at the time: whether a correlation that clean was too good to be true.** It was, in the sense that it was expected rather than meaningful: two quantities derived from the same model will naturally agree almost exactly, and that agreement is not, on its own, evidence that the recommendation holds up in the real world. The same discipline, treating an unusually clean result as a prompt to check for circularity rather than as confirmation, applies directly to the `high_risk` proxy above.

Practical implication for any future extension of this project: **if a real accident dataset became available, the existing proxy trained models should not simply be relabelled or fine-tuned against the new target and assumed to still be valid. They should be retrained and reevaluated from a clean baseline,** since a model shaped around reproducing the proxy's own definition may carry that shape into a supposedly unbiased retraining in ways that are not obvious from the code alone.

1.3 Uneven error distribution across conditions and time periods

Every regression model in this project is evaluated with a single overall MAE and R squared on a chronological holdout, which by construction blends easy and hard cases. Task 6's drift monitoring script, `drift_monitor.py`, gives a direct, measured example of how misleading that single number can be. A batch of six requests deliberately including a snow event, a thunderstorm and a holiday produced a batch MAE of 920.79, 3.43 times higher than the production model's usual holdout MAE of 268.78. **The model's real error on an unusual but entirely plausible combination of conditions was more than three times worse than its headline accuracy figure suggests.**

This follows directly from the coverage limitations in section 1.1: conditions that are rare in training, severe weather, holidays, heavy precipitation, are exactly the conditions where the model has seen the least evidence, so above average error there is expected, and this monitoring run gives a concrete, reproducible measurement of that rather than leaving it as a theoretical concern.

Time of day and day of week are also unevenly represented. Weekday and weekend traffic patterns were shown throughout Parts 2 and 3 to differ substantially, the CLI app's weekday versus weekend comparison and the recommendation system's separate weekday and weekend windows both treat this as a first order distinction, so a model evaluated only in aggregate could look accurate overall while being systematically off for one of the two regimes. **The recommendation system's final, rush hour anchored window selection logic was itself the product of**

three earlier approaches, each failing in a different way for a different day type or weather combination (a search boundary artefact, then a false positive on a trivial wiggle, then a false negative on a genuine but flattened dip), **which is first hand evidence that error and reliability are not uniform across conditions in this problem.**

The precipitation fields, rain_1h and snow_1h, are dominated by zero readings, with a small number of unusually wet or snowy hours sitting far out in the tail of an otherwise almost constant distribution. Task 6's drift monitor ran directly into this: a check based on the mean and standard deviation of snow_1h produced an extremely large deviation score from a single ordinary looking snow request, purely because the training standard deviation for that field is so small. The lesson generalises beyond this one monitoring threshold: **any error or fairness analysis that summarises these fields with a mean and a standard deviation, risks understating how differently the model behaves on the rare, high precipitation hours compared with the overwhelmingly dry majority.**

2. Governance and Sustainability

2.1 Oversight before real world decision making

This project's MLflow Model Registry, built in Task 6.1 and 6.2, already provides the mechanical building block that any real deployment would need: a single named, versioned pointer, the production alias, to whichever model is currently live, kept separate from the many exploratory runs sitting alongside it. **What it does not yet provide is a governance process around that pointer.** In this project the alias moves automatically whenever model_versioning.py is run, with no approval step. **Before a model like this influenced any real decision, moving the production alias should require a deliberate, logged approval,** a named person or a defined committee signing off, rather than happening as a side effect of a script running.

Given the proxy label risk in section 1.2, any oversight process should include a domain specialist, not only a data or software team. **The central open question here, whether congestion combined with bad weather is a reasonable stand in for accident risk, is a road safety and transport planning question, not a modelling question, and this project is not positioned to answer it alone.**

The deployment simulation in Task 6.3 has no authentication, no rate limiting, and no auditing beyond the prediction log it happens to keep. That is acceptable for a course simulation but would need to change before any real deployment, so that every prediction which could inform a real decision is attributable to a request and a caller, and so that the service cannot be queried or overloaded by anyone with network access to it.

The drift monitoring built in Task 6.4 and 6.5 is a starting point for ongoing oversight, not a finished one. Its thresholds were sized, by design and documented directly in the script's own docstring, for demonstrating the mechanism against a handful of test requests rather than for running unattended against a real, continuous stream of traffic. **Before relying on it operationally, the thresholds would need retuning against a much larger rolling window, and an explicit escalation path** would need defining for what happens when an alert fires, for example a named reviewer, and the ability to move the production alias back to a previous registered version if a newly promoted one turns out to be underperforming.

The prediction error check inside that monitoring script can currently only run when a live request happens to match a timestamp already present in the historical dataset, which will not be true of any genuinely new request once this moves beyond a simulation. **A real oversight process needs a plan for closing that loop, some way of eventually observing the true traffic volume for past predictions, so that live accuracy keeps being measured after deployment, not only at the point the model was first validated.**

2.2 Environmental and resource trade offs

The clearest, most concrete resource trade off in this project is documented directly in Task 6's model versioning history, summarised in Table 1.

Version	Hyperparameters	Serialized size	Test MAE	Test R2
v1	n_estimators=300, max_depth=None,	882.1 MB	300.05	0.9296

(unconstrained)	min_samples_leaf=1			
v2 (milder cap)	n_estimators=200, max_depth=20, min_samples_leaf=5	92.1 MB	274.48	0.9395
v3 (production)	n_estimators=150, max_depth=15, min_samples_leaf=10	30.7 MB	268.78	0.9411

An unconstrained random forest, grown with scikit learn's own defaults and 300 trees, serialised to 882.1 MB and took about 6.5 seconds to train, yet scored worse on the holdout test set, MAE 300.05, than the much smaller version that was kept in production. That final version, capped at a maximum depth of 15 and a minimum of 10 samples per leaf, serialised to 30.7 MB, well under 4% of the uncapped model's size, and scored better rather than worse, MAE 268.78. **Every megabyte of that difference has a real cost, in the memory needed to load the model, the storage needed to keep it and every earlier version, and the compute needed to run it repeatedly in a deployed service.** This project's own experience argues that chasing the largest, least constrained model was not only unnecessary here but was measurably counterproductive on the environmental and the accuracy axis at the same time.

The same logic shaped the decision not to log the two larger historical versions as MLflow artefacts. Task 6.1 and 6.2 log v1 and v2's parameters and metrics for the historical record but deliberately do not attach the 882 MB and 92 MB model files themselves, so that keeping an audit trail of how the current model was reached does not multiply this project's storage footprint by carrying every superseded, oversized model forward indefinitely.

The choice of model family in Task 3 reflects a similarly proportionate default. The deep learning component uses a comparatively small LSTM, a single 64-unit layer with dropout and a small dense head, trained for at most 30 epochs with early stopping, rather than a larger sequence model. Given the dataset's size, on the order of tens of thousands of hourly rows, and the accuracy already achieved, an R squared consistently above 0.98 across repeated runs, a larger architecture would very likely have used substantially more training compute without a corresponding gain in accuracy.

MLflow itself runs here against a local SQLite file and a local artefact folder rather than a hosted tracking server or cloud object storage, proportionate to a single developer capstone running on one machine. **This convenience has already produced a small, real illustration of unmanaged growth: repeated runs of mlflow_tracking.py and model_versioning.py have left the mlflow/artifacts folder at roughly 100 MB,** made up of several duplicate copies of the same production sized model rather than one authoritative copy. At this project's scale that is a minor housekeeping item, but **it previews a real cost that any team running frequent retraining against an experiment tracker needs a deliberate retention policy for,** deciding how many historical versions are worth keeping against the storage, and ultimately energy, cost of keeping all of them permanently.

Taken together, the pattern across this project's modelling choices, capping tree depth and leaf size, keeping the LSTM small, choosing a local rather than a hosted tracking backend, and declining to store oversized historical artefacts, was in each case driven primarily by a practical constraint, GitHub's file size limits and a single machine environment, rather than an explicit environmental objective. The fact that the more frugal choice also matched or beat the more resource intensive alternative on accuracy, in the one case where both were directly compared, the regressor's three versions, is worth stating plainly as a finding rather than an assumption: at this problem's scale, restraint was not a trade off against quality.

Summary

The points raised across this report are not generic disclaimers; each is tied to a specific figure, script, or moment in this project's own development: a 23% data gap of unknown cause, a proxy label built partly from its own input features, a measured 3.43 times error increase on rare conditions, and a 29 times difference in model size that came with better rather than worse accuracy. **Read together, they argue for the same conclusion from three different directions: this project demonstrates a complete and internally consistent machine learning workflow, but the high_risk proxy label in particular means its outputs should not be used to inform real safety or resourcing decisions** without validation against real accident data and review by a road safety specialist, and any move toward real

deployment should bring the approval, auditing, and monitoring practices in section 2.1 up to the same standard as the modelling work itself.